Homework: Privacy-Preserving AI Bounty Judge

Extend the workshop app so submissions stay hidden until judging is complete.

Format	Individual or teams of 2
Focus	Smart contract architecture, hidden submissions, and Ritual-enabled private judging
Required Track	Commit-reveal submission flow
Advanced Track	Ritual-native encrypted submissions judged inside a TEE
Main Question	How can a bounty app prevent answer copying before judging?

1. Context

In the workshop, we built a simple AI Bounty Judge: the bounty owner creates a bounty, participants submit answers, Ritual AI judges all submissions after the deadline, and the owner finalizes one winner.

However, the workshop version has an important weakness: answers are public immediately after submission. This means later participants can read earlier answers, copy useful ideas, and submit an improved version. That is unfair in a bounty system where only one person can win.

Homework Goal

Modify the system so answers stay hidden during the submission phase and are revealed only after judging is complete.

2. Learning Objectives

- Understand the difference between public on-chain state and hidden commitments.
- Implement a commit-reveal workflow in Solidity.
- Reason about fairness problems in bounty and competition contracts.
- Compare a generic blockchain solution with a Ritual-native private execution solution.
- Design a system where AI can judge submissions without giving participants an unfair information advantage.

3. Required Track: Commit-Reveal Bounty

Implement a commit-reveal version of the AI Bounty Judge contract. This is the required homework track. It is a smart-contract-focused solution that works on any EVM chain.

Required Flow

1. Bounty owner creates a bounty with a reward, submission deadline, and reveal deadline.
2. Participants submit only a commitment hash during the submission phase.

3. The real answers are not public yet.
4. During the reveal phase, participants reveal their answer and salt.
5. The contract verifies that keccak256(answer, salt, sender, bountyId) matches the original commitment.
6. Only valid revealed answers are eligible for AI judging.
7. After the reveal deadline, the owner calls judgeAll().
8. Ritual AI judges all revealed answers together.
9. Owner finalizes one winner and the contract pays the reward.

Suggested Commitment Formula

```
bytes32 commitment = keccak256(
    abi.encodePacked(answer, salt, msg.sender, bountyId)
);
```

Including **msg.sender** and **bountyId** helps prevent another participant from copying a commitment and revealing someone else's answer.

Required Solidity Functions

```
function submitCommitment(uint256 bountyId, bytes32 commitment) external;

function revealAnswer(
    uint256 bountyId,
    string calldata answer,
    bytes32 salt
) external;

function judgeAll(uint256 bountyId, bytes calldata llmInput) external;

function finalizeWinner(uint256 bountyId, uint256 winnerIndex) external;
```

Required Contract Rules

- Participants can submit commitments only before the submission deadline.
- Participants can reveal answers only after the submission deadline and before the reveal deadline.
- A participant can submit only one commitment per bounty.
- A reveal is valid only if the commitment hash matches.
- Unrevealed submissions are not eligible for judging.
- The owner can judge only after the reveal deadline.
- The owner can finalize only after judging is complete.
- Only one winner receives the bounty reward.

4. Advanced Track: Ritual-Native Hidden Submissions

The commit-reveal flow is useful, but it has one limitation: answers become public before AI judging happens. The advanced track asks you to design or implement a more Ritual-native version where encrypted answers can stay hidden until the AI judging step is complete.

Advanced Flow

1. Each participant encrypts their answer for a Ritual TEE executor or Ritual privacy/key flow.

2. The contract stores only encrypted submissions or references to encrypted submissions.
3. Before judging, other participants cannot read the plaintext answers.
4. During judgeAll(), the TEE-based workflow decrypts the answers privately and sends them to the LLM for comparison.
5. After judging, the system reveals the winner and all answers together, or publishes a reference and hash to the revealed answer bundle.

Advanced Design Requirements

- Explain where the plaintext answers exist and who can read them.
- Explain what is stored on-chain and what is stored off-chain.
- Explain how the LLM receives all submissions together.
- Explain how the final reveal happens.
- Explain how the contract verifies or commits to the final revealed bundle.
- Avoid storing large plaintext answers directly on-chain unless you justify the gas cost.

Suggested Reveal Pattern

Instead of storing all plaintext answers directly in contract storage, publish a revealed answers bundle off-chain and store its hash on-chain. For example: revealedAnswersRef plus revealedAnswersHash.

Example Final Output Shape

```
{
  "winnerIndex": 2,
  "ranking": [{ "index": 2, "score": 94, "reason": "Best satisfies the rubric." }],
  "revealedAnswersRef": "ipfs://... or storage-ref://...",
  "revealedAnswersHash": "0x...",
  "summary": "Submission 2 is the strongest answer."
}
```

5. Ritual Feature Focus

Your design should use Ritual for more than simply calling an LLM. The advanced track should think about Ritual's TEE-backed execution, encrypted secrets/private inputs, and the idea that AI can evaluate data that should not be public before the judging phase.

- TEE-backed execution: the judging logic can run in an environment that sees private inputs while keeping them hidden from the public chain.
- Encrypted inputs/secrets: participant data or storage credentials should not appear in plaintext on-chain.
- Batch judging: all submissions should be judged together in one AI request, not one LLM call per answer.
- Human-in-the-loop finalization: the AI can recommend a winner, but the owner still finalizes the payout.

6. Deliverables

- Updated Solidity contract implementing the required commit-reveal flow.
- A short README explaining the new bounty lifecycle.

- At least one test or test plan covering valid and invalid reveal cases.
- A short architecture note comparing commit-reveal and Ritual-native encrypted submissions.
- If you attempt the advanced track, include a diagram or explanation of the private submission flow.

7. Evaluation Criteria

Category	What We Look For	Weight
Commit-reveal correctness	Commitments, reveals, deadlines, and eligibility are enforced correctly.	30%
Smart contract safety	Access control, payout logic, and invalid states are handled safely.	20%
Ritual understanding	The solution respects batch judging and explains how Ritual can support private AI evaluation.	20%
Code clarity	The contract is readable, well-structured, and easy to reason about.	15%
Testing / explanation	Tests or a clear test plan demonstrate important edge cases.	15%

8. Reflection Question

Answer this in 5-8 sentences:

What should be public, what should stay hidden, and what should be decided by AI versus by a human in a bounty system?

9. Notes and Constraints

- Do not call the LLM once per submission inside a Solidity loop. Use one batch judging request.
- Do not reveal answers during the submission phase.
- Do not automatically pay a winner from AI output unless you clearly explain how the result is parsed and validated.
- Keep the required track simple. The advanced track can be a design document if full implementation is too complex.

End of homework statement